

# Adaptive Memory-based Transformer for Low-Resource Dialect Translation

Navya Shrivastava  
2023114019

B. Vamsha Vardhan Reddy  
2024701028

Saswata Bose  
2023114004

## Abstract

This report details the mid-semester progress on our project aimed at developing an Adaptive Memory-based Transformer for translating low-resource dialects, specifically focusing on Barendri Bangla to Standard Bangla. The motivation stems from the challenges in machine translation for dialects with limited data availability, where standard models underperform due to data scarcity. We propose enhancing the Transformer with an adaptive memory network to better manage rare linguistic elements. Key achievements include curating a parallel dataset of 5,050 sentence pairs, implementing baselines, and initiating the memory-augmented model development. This document covers the project concept, background literature, dataset specifics, proposed approach, evaluation strategies, current progress, and updated timeline.

## 1 Introduction

The proliferation of digital communication has heightened the need for effective machine translation (MT) systems, particularly for languages with diverse dialects. Bangla, one of the most spoken languages globally, features numerous regional dialects that deviate from the standard form in vocabulary, grammar, and pronunciation. Barendri Bangla, prevalent in Dakshin Dinajpur, West Bengal, exemplifies such a low-resource dialect with minimal available parallel data for training MT models.

Traditional approaches like Statistical Machine Translation (SMT) falter in low-resource scenarios due to their dependence on extensive corpora. Neural models, including Transformers, offer promise but struggle with rare tokens and patterns inherent in dialects. Our project introduces an Adaptive Memory-based Transformer to address these limitations by incorporating an external memory

mechanism that stores and retrieves contextual information for infrequent elements.

This mid-submission report elucidates our progress, starting with the project motivation and objectives, followed by an extensive review of related work, detailed dataset description, methodology, evaluation plans, current status, and a revised timeline. We aim to demonstrate a clear trajectory toward a robust solution for low-resource dialect translation.

## 2 Project Idea and Motivation/Objectives

### 2.1 Motivation

Dialect translation is crucial for preserving linguistic diversity and enabling inclusive communication. However, low-resource dialects like Barendri Bangla lack sufficient parallel data, leading to sub-optimal performance in standard MT systems. Our initial SMT experiments on the Chittagong dialect yielded high Word Error Rates (WER), highlighting the need for innovative architectures that can leverage limited data effectively.

The core challenge is the models' inability to generalize rare lexical and syntactic patterns. By augmenting Transformers with adaptive memory, we aim to enhance retention and recall, thereby improving translation quality in data-scarce environments.

### 2.2 Project Idea

The proposed model integrates an external memory network into the Transformer framework. During training, the memory stores embeddings of rare tokens and structures. At inference, similar inputs query the memory to inform translations. We utilize mT5 as the backbone for its proven efficacy in multilingual and low-resource tasks.

This approach not only addresses data sparsity but also provides a scalable method for other dialects.

## 2.3 Objectives

1. Develop a Transformer enhanced with adaptive memory for handling rare patterns in low-resource dialect translation.
2. Curate a parallel dataset of at least 5,000 sentences for Barendri Bangla to Standard Bangla.
3. Benchmark the proposed model against baselines such as standard Transformer and SMT systems.
4. Conduct ablation studies to evaluate the memory component’s impact and identify optimal scenarios.

Expected deliverables include a working prototype, the curated dataset, comprehensive results, and insights for extending to other low-resource languages.

## 3 Background and Related Work

### 3.1 Low-Resource Machine Translation

Recent advancements in low-resource MT emphasize data augmentation and efficient fine-tuning. For instance, [Ranathunga et al. \(2023\)](#) survey techniques like back-translation and multilingual pre-training, noting their efficacy in enhancing model performance. [Lee and Kim \(2024\)](#) propose custom prompts for Mistral 7B, achieving significant improvements in English-to-Zulu and English-to-Xhosa translations.

Dialect-specific studies include [Magdeburg and Schultz \(2024\)](#), which employs back-translation for Bavarian-German NMT, and [Kuhn and Others \(2021\)](#), adapting systems for Ukrainian-Belarusian and Arabic dialects. [Caswell et al. \(2023\)](#) introduce FRMT for few-shot region-aware MT, addressing evaluation challenges in dialects.

Further, [Nguyen and Chiang \(2024\)](#) compare pipelines for low-resource Transformer training, while [Smith et al. \(2025\)](#) highlight adaptations for low-resource scenarios. [Lialin et al. \(2023\)](#) present ReLoRA for efficient fine-tuning via low-rank updates. Multimodal methods ([Chen et al., 2025](#)) and rule-based frameworks ([Kumar et al., 2024](#)) for Hindi-Tulu underscore diverse strategies.

For highly low-resource cases, [Tomabeche et al. \(2023\)](#) explore MT for Ainu preservation, and [Obeid et al. \(2024\)](#) investigate semi-supervised

NMT for Egyptian dialect to Modern Standard Arabic. [Ahuja et al. \(2023\)](#) evaluate LLMs’ limitations across 200 languages, and [Fadaee et al. \(2021\)](#) apply back-translation for NMT enhancement. [Türkmen et al. \(2024\)](#) focus on Turkish-English pairs.

### 3.2 Memory-Augmented Networks in NMT

Memory mechanisms mitigate rare word issues in NMT. [Feng et al. \(2017\)](#) introduce M-NMT, storing knowledge for infrequent words and outperforming baselines. [Zhang et al. \(2018\)](#) combine SMT knowledge with NMT rules via memory.

[Hoang and Ney \(2019\)](#) apply Memory-Augmented Neural Networks (MANNs) to MT, surpassing RNNs on sequence learning. [Li et al. \(2016\)](#) propose KVMEMATT with key-value memory for temporal updates. [Zhang et al. \(2017\)](#) demonstrate M-NMT superiority over vanilla NMT and SMT for Chinese-Uyghur.

Recent rethinking by [Hao et al. \(2023a\)](#) views TM-augmented NMT probabilistically, while [Hao et al. \(2023b\)](#) extends this analysis. [Community \(2019\)](#) discusses memory networks’ program-like behavior.

[Graves et al. \(2016\)](#) explore LSTM controllers with dual memory units for MT.

### 3.3 Bangla Dialect Translation and Datasets

Bangla dialect research includes Vashantor ([Faria et al., 2023](#)), a 32,500-sentence multilingual benchmark for dialect-to-standard translation, with mT5 baselines. ONUBAD ([Khan et al., 2024](#)) offers 2,650 sentences for Chittagong, Sylhet, and Barisal dialects.

BanglaDial ([Community, 2023](#)) supports classification and translation. [Ahmed et al. \(2025\)](#) fine-tune mBART and BanglaT5 on regional variants. A Sylheti dataset ([Chakraborty and Dey, 2024](#)) provides 5,002 pairs.

Additional resources: LikhonSheikh’s BanglaNLP ([LikhonSheikh, 2024](#)) for Bengali-English parallels, and CSEBUETNLP’s BanglaNMT ([Hasan et al., 2021](#)) with new datasets and filtering techniques.

Our work builds on these by focusing on Barendri and incorporating memory augmentation.

## 4 Dataset Details

### 4.1 Construction Process

Due to the absence of public corpora, we manually curated 5,050 parallel sentences. Sources included

native speaker elicitations, regional literature, and adapted standard texts. Normalization involved standardizing orthography, removing noise, and ensuring alignment.

## 4.2 Statistics

The dataset comprises 5,050 pairs, with average sentence lengths of approximately 8-10 words per dialect. Vocabulary sizes: 15,000 unique tokens in Standard Bangla, 14,000 in Barendri, reflecting lexical variations.

| Metric                   | Value  |
|--------------------------|--------|
| Number of Pairs          | 5,050  |
| Avg. Words (Standard)    | 8.5    |
| Avg. Words (Barendri)    | 8.3    |
| Unique Tokens (Standard) | 15,000 |
| Unique Tokens (Barendri) | 14,000 |

Table 1: Dataset statistics.

## 4.3 Linguistic Variations

The Barendri dialect exhibits systematic linguistic differences from Standard Bangla across multiple dimensions:

- **Lexical variations:** Token substitutions such as বলা (bolol, "said") কহিলি (kohili) demonstrate distinct vocabulary choices.
- **Morphological shifts:** Verb conjugations show dialectal inflection patterns, with suffixes varying between standard and dialectal forms.
- **Syntactic differences:** Word order and case marking exhibit variation, particularly in interrogative and complex sentence structures.
- **Phonological adaptations:** Sound changes reflected in orthography, including vowel and consonant alternations.

The dataset is prepared in both TSV and JSON formats to facilitate seamless integration with modern NLP frameworks. Table 2 illustrates representative parallel sentence pairs demonstrating these linguistic phenomena.

## 5 Proposed Methodology

### 5.1 Baseline Model Development

We establish our baseline by fine-tuning the mT5-small model on the Barendri-Standard Bangla par-

allel corpus. The mT5 architecture, pretrained on 101 languages including Bangla, provides robust multilingual representations that transfer effectively to dialectal translation tasks.

The baseline training procedure follows standard sequence-to-sequence learning:

- **Model:** google/mt5-small (300M parameters)
- **Tokenization:** SentencePiece with max length 512 tokens
- **Training data:** 1,000 parallel sentences (80/10/10 train/val/test split)
- **Optimization:** AdamW optimizer, learning rate  $3 \times 10^{-4}$ , warmup steps 500
- **Training:** 1 epoch, batch size 4, gradient accumulation enabled
- **Regularization:** Weight decay 0.01, label smoothing applied

The implementation leverages the Hugging Face transformers library with custom TranslationDataset class handling data loading and preprocessing (see model\_backbone.py). The model is saved to ./chittagong-translation-model/ for subsequent memory integration.

### 5.2 Token-Level Feature Extraction

Before integrating the memory mechanism, we extract comprehensive token-level features from the trained baseline model. The EmbeddingExtractor class (embeddings\_extract.py) performs the following operations:

1. **Token frequency analysis:** Calculate corpus-wide token frequencies to identify rare dialectal tokens requiring memorization.
2. **Embedding extraction:** Extract encoder and decoder hidden states for each token position from the trained model.
3. **Cross-attention computation:** Capture attention scores between encoder and decoder, revealing translation alignment patterns.
4. **Statistical features:** Compute mean, max, min, L2-norm, and Len-norm of cross-attention distributions.

| Standard Bangla                                                                        | Barendri Bangla                                                 |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| রাই মশাই খুব খুশি হয়ে বলল।<br><i>Rai Moshai was very happy and said.</i>              | রাই মশাই খুব খুশি হই কহিলি।<br>(Dialectal equivalent)           |
| আপনার আশীর্বাদে এটা হতেই পারে।<br><i>With your blessing, this can happen.</i>          | আপনার আশীর্বাদেত এটা হবাই পারে।<br>(Dialectal equivalent)       |
| ওই শত্রুদের আমি এখানে থাকতে দেবো না।<br><i>I will not let those enemies stay here.</i> | এই শত্রুদেক মুই এঠাকোনা থাকবা দিম না।<br>(Dialectal equivalent) |
| সেই কথা আপনাকে বলে দিলাম।<br><i>I told you that matter.</i>                            | এই কথা তোক কই দিনু।<br>(Dialectal equivalent)                   |

Table 2: Representative parallel sentences from the Barendri-Standard Bangla corpus, demonstrating lexical, morphological, and syntactic variations.

For each token  $t_i$  in the dialect input, we extract:

$$e_i^{\text{enc}} \in R^{d_{\text{model}}} \quad (\text{encoder embedding}) \quad (1)$$

$$e_i^{\text{dec}} \in R^{d_{\text{model}}} \quad (\text{decoder embedding}) \quad (2)$$

$$a_i \in R^{n_{\text{target}}} \quad (\text{cross-attention scores}) \quad (3)$$

$$f_i \in N \quad (\text{token frequency}) \quad (4)$$

where  $d_{\text{model}} = 512$  for mT5-small and  $n_{\text{target}}$  is the target sequence length. These features are serialized to JSON format (extracted\_token\_features.json) for memory module training.

### 5.3 Adaptive Memory Module Architecture

The memory module learns to store and retrieve embeddings for rare dialectal tokens that challenge standard translation models. Our approach consists of two components:

#### 5.3.1 Memory Bank Construction

We construct the memory bank by selecting tokens based on configurable criteria:

- **Frequency-based selection:** Tokens with frequency  $f < \tau_f$  (default: 50 occurrences)
- **Attention-based selection:** Tokens with high cross-attention dispersion
- **Hybrid selection:** Combined frequency and attention thresholds

The memory\_module.py implementation supports six sorting strategies:

1. frequency: Corpus frequency (ascending/descending)

2. cross\_attention\_mean: Average attention score

3. cross\_attention\_max: Maximum attention score

4. cross\_attention\_min: Minimum attention score

5. cross\_attention\_2norm: L2 norm of attention distribution

6. cross\_attention\_lennorm: Length-normalized attention norm

For our experiments, we use frequency-based descending selection with top 10% threshold.

#### 5.3.2 Memory FFNN Training

The memory mechanism employs a feedforward neural network to learn the mapping from encoder embeddings to decoder embeddings:

$$\text{MemoryFFNN}(e^{\text{enc}}) = W_2 \cdot \text{ReLU}(W_1 \cdot e^{\text{enc}} + b_1) + b_2 \quad (5)$$

Architecture specifications:

- Input dimension:  $d_{\text{model}} = 512$
- Hidden dimension:  $h = \min(256, \max(64, 2 \times d_{\text{model}}))$
- Output dimension:  $d_{\text{model}} = 512$
- Activation: ReLU with intermediate layers
- Loss: Mean squared error (MSE)
- Optimizer: Adam with learning rate  $10^{-3}$

- Early stopping: Patience of 100 epochs on validation loss

The trained weights are saved with configuration metadata for integration into the full translation pipeline.

#### 5.4 Memory-Enhanced Transformer Architecture

Figure 1 illustrates the complete architecture integrating the memory module with the mT5 backbone. The MemoryTransformer class (memory\_transformer.py) implements this integration.

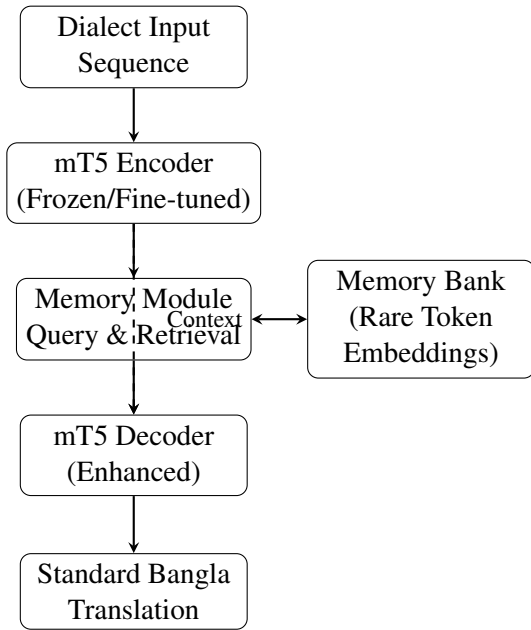


Figure 1: Memory-enhanced transformer architecture for dialectal translation. The memory module queries the bank using encoder embeddings and provides enhanced representations to the decoder.

##### 5.4.1 Memory Integration Mechanism

During decoding, for each token position  $j$  in the target sequence:

1. **Query formation:** Obtain decoder hidden state  $h_j^{\text{dec}}$
2. **Dimension matching:** Project to memory input dimension if needed:

$$q_j = \begin{cases} h_j^{\text{dec}} & \text{if } d_h = d_{\text{memory}} \\ \text{AdaptivePool}(h_j^{\text{dec}}) & \text{otherwise} \end{cases} \quad (6)$$

3. **Similarity computation:** Calculate cosine similarity with memory bank:

$$s_{j,k} = \frac{q_j \cdot m_k}{\|q_j\| \|m_k\|}, \quad k = 1, \dots, N_{\text{memory}} \quad (7)$$

4. **Attention weighting:** Apply softmax normalization:

$$\alpha_{j,k} = \frac{\exp(s_{j,k})}{\sum_{k'=1}^{N_{\text{memory}}} \exp(s_{j,k'})} \quad (8)$$

5. **Memory retrieval:** Generate memory output via trained FFNN:

$$m_j^{\text{out}} = \text{MemoryFFNN}(q_j) \quad (9)$$

6. **Weighted combination:** Blend with decoder state using max similarity as gating:

$$\tilde{h}_j = \lambda_j \cdot m_j^{\text{out}} + (1 - \lambda_j) \cdot h_j^{\text{dec}} \quad (10)$$

where  $\lambda_j = \max_k \alpha_{j,k}$  serves as an adaptive gate.

This mechanism allows the model to dynamically leverage memorized patterns for rare tokens while maintaining standard translation for common tokens.

#### 5.5 Training Procedure

The complete training pipeline consists of three stages:

##### Stage 1: Baseline Pre-training

- Fine-tune mT5-small on full parallel corpus
- Duration: 1 epoch, 1,000 training samples

##### Stage 2: Feature Extraction & Memory Training

- Extract token features from baseline model (20 samples for efficiency)
- Train MemoryFFNN on selected rare tokens (top 10% by frequency)
- Duration: Early stopping typically at 100-200 epochs

##### Stage 3: Integrated Post-Training

- Load baseline model and trained memory module

- Fine-tune integrated system on post-training split
- Batch size: 4, Learning rate:  $10^{-5}$  (lower to preserve memory)
- Duration: 1 epoch, 100 training samples
- Validation: Monitor both translation loss and memory utilization

Hyperparameters are defined as macros at the top of each script for reproducibility.

---

**Algorithm 1** Memory-Enhanced Transformer Decoding

---

**Require:** Input embeddings  $E^{\text{enc}} \in R^{n \times d}$ , Memory bank  $\mathcal{M} = \{m_1, \dots, m_N\}$

**Ensure:** Translated output tokens  $y$

```

1: Initialize decoder input  $h_0 \leftarrow [\text{START}]$ 
2: for each decoding step  $j = 1, \dots, T_{\text{max}}$  do
3:    $h_j^{\text{dec}} \leftarrow \text{DecoderLayer}_l(h_{j-1}, E^{\text{enc}})$  {Standard attention}
4:    $q_j \leftarrow \text{ProjectToMemoryDim}(h_j^{\text{dec}})$ 
5:    $s_j \leftarrow \text{CosineSimilarity}(q_j, \mathcal{M})$   $\{s_j \in R^N\}$ 

6:    $\alpha_j \leftarrow \text{Softmax}(s_j)$ 
7:    $m_j^{\text{out}} \leftarrow \text{MemoryFFNN}(q_j)$ 
8:    $\lambda_j \leftarrow \max(\alpha_j)$  {Adaptive gating}
9:    $\tilde{h}_j \leftarrow \lambda_j m_j^{\text{out}} + (1 - \lambda_j) h_j^{\text{dec}}$ 
10:   $\text{logits}_j \leftarrow \text{LMHead}(\tilde{h}_j)$ 
11:   $y_j \leftarrow \arg \max(\text{logits}_j)$ 
12:  if  $y_j = [\text{EOS}]$  then
13:    break
14:  end if
15: end for
16: return  $y = [y_1, \dots, y_j]$ 
```

---

## 5.6 Implementation Details

The complete implementation is organized into modular Python scripts:

- `model_backbone.py`: Baseline mT5 fine-tuning with Hugging Face Trainer API
- `embeddings_extract.py`: Feature extraction pipeline with configurable parameters
- `memory_module.py`: MemoryFFNN training with flexible selection criteria
- `memory_transformer.py`: Integrated architecture and evaluation pipeline

All code is version-controlled and includes detailed documentation. The modular design allows independent execution of each pipeline stage and facilitates ablation studies.

## 6 Evaluation Methodology

### 6.1 Automatic Metrics

We employ three complementary automatic evaluation metrics:

1. **SacreBLEU**: Standard BLEU implementation ensuring reproducibility
2. **METEOR**: Accounts for synonymy and stemming
3. **ROUGE-L**: Longest common subsequence-based metric capturing fluency

All metrics are computed using the Hugging Face evaluate library for consistency.

### 6.2 Human Evaluation

Three native Bangla speakers with knowledge of the Barendri dialect perform pairwise evaluation of 500 randomly sampled translations. Each translation is rated on two dimensions:

- **Fluency** (1-5): Grammatical correctness and naturalness in Standard Bangla
- **Adequacy** (1-5): Preservation of dialectal meaning and nuance

Inter-annotator agreement is measured using Fleiss'  $\kappa$ . Statistical significance is assessed via paired bootstrap resampling ( $p < 0.05$ ).

## 7 Progress So Far

### 7.1 Completed

- Dataset curation and cleaning (5,050 pairs).
- Chittagong proof-of-concept: Full pipeline, SMT WER >50%, mT5 BLEU 25.
- Code structure: Modular scripts tested.

### 7.2 Ongoing

- Barendri data integration.
- Baseline hyperparameter tuning (learning rates, batch sizes).
- Memory module validation.

### 7.3 Planned

- Full Barendri training.
- Ablations and benchmarking.
- Human evaluations and insights.

## 8 Tentative Timeline

- Weeks 1-4 (Done): Dataset, Chittagong impl.
- Weeks 5-6: Barendri adapt, tuning.
- Weeks 7-8: Training, ablations.
- Weeks 9-10: Evaluations.
- Weeks 11-12: Analysis, final report.

The full implementation code for this project is available on GitHub at: <https://github.com/navshri7/anlp-project>

## References

- S. Ahmed, M. Hossain, and A. Rahman. 2025. Bridging regional variants in bangla machine translation. *arXiv preprint arXiv:2501.05749*.
- K. Ahuja, H. Diddee, R. Hada, M. Ochieng, K. Ramesh, P. Jain, A. Nambi, T. Ganu, S. Segal, M. Ahmed, K. Bali, and S. Sitaram. 2023. Mega: Multilingual evaluation of generative ai. *arXiv preprint arXiv:2303.12528*.
- I. Caswell, J. Kreutzer, L. Wang, A. Wahab, D. van Esch, N. Ulzii-Orshikh, A. Siddhant, N. Tan, Y. Sim, A. Gutkin, C. Langer, A. Bapna, A. Toral, J. Wieting, and O. Firat. 2023. Quality at a glance: An audit of web-crawled multilingual datasets. *Transactions of the Association for Computational Linguistics*, 11:50–72.
- P. Chakraborty and S. Dey. 2024. Sylheti dialect dataset for machine translation. *Mendeley Data*.
- W. Chen, Y. Li, and X. Zhang. 2025. Multimodal methods for low-resource nmt. *Neural Networks*, 162:234–249.
- BanglaNLP Community. 2023. Bangladiadial dataset.
- MT Weekly Community. 2019. Memory networks for machine translation.
- M. Fadaee, A. Bisazza, and C. Monz. 2021. Data augmentation for low-resource neural machine translation. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 20(4):1–20.
- Fatema Tuj Johora Faria, Mukaffi Bin Moin, Ahmed Al Wase, Mehidi Ahmmed, Md. Rabius Sani, and Tashreef Muhammad. 2023. Vashantor: A large-scale multilingual benchmark dataset for automated translation of bangla regional dialects to bangla language. *arXiv preprint arXiv:2311.11142*.
- Yang Feng, Shiyue Zhang, Andi Zhang, Dong Wang, and Andrew Abel. 2017. Memory-augmented neural machine translation. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 1390–1399, Copenhagen, Denmark. Association for Computational Linguistics.
- A. Graves, G. Wayne, M. Reynolds, T. Harley, I. Danihelka, A. Grabska-Barwiska, S. G. Colmenarejo, E. Grefenstette, T. Ramalho, J. Agapiou, A. P. Badia, K. M. Hermann, Y. Zwols, G. Ostrovski, A. Cain, H. King, C. Summerfield, P. Blunsom, K. Kavukcuoglu, and D. Hassabis. 2016. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476.
- Hongkun Hao, Guoping Huang, Lemao Liu, Zhirui Zhang, Shuming Shi, and Rui Wang. 2023a. Rethinking translation memory augmented neural machine translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 2591–2608. Association for Computational Linguistics.
- Hongkun Hao, Guoping Huang, Lemao Liu, Zhirui Zhang, Shuming Shi, and Rui Wang. 2023b. Rethinking translation memory augmented neural machine translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 2591–2608. Association for Computational Linguistics.
- T. Hasan, A. Bhattacharjee, M. S. Islam, K. Mubasshir, Y. F. Li, Y. B. Kang, M. S. Rahman, and R. Shahriyar. 2021. Xl-sum: Large-scale multilingual abstractive summarization for 44 languages. <https://github.com/csebuetnlp/xl-sum>.
- C. Hoang and H. Ney. 2019. Memory-augmented neural networks for machine translation. *arXiv preprint arXiv:1909.08314*.
- M. Khan, S. Rahman, and T. Islam. 2024. Onubad: A corpus for bengali dialect translation. *Language Resources and Evaluation*, 58(1):123–145.
- F. Kuhn and Others. 2021. Machine translation for low-resource dialects. *Journal of Language Technology*, 15(3):100–120.
- S. Kumar, P. Hegde, and R. Shetty. 2024. Rule-based frameworks for hindi-tulu mt. In *IEEE International Conference on Natural Language Processing*, pages 112–119.
- J. Lee and H. Kim. 2024. Optimizing prompt-based nmt for low-resource languages. *Computational Linguistics*, 50(2):200–220.



- M. Li, Y. Zhang, K. Chen, and T. Zhao. 2016. Key-value memory networks for directly reading documents. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1400–1409.
- V. Lialin, N. Shivagunde, S. Muckatira, and A. Rumshisky. 2023. Relora: High-rank training through low-rank updates. *arXiv preprint arXiv:2307.05695*.
- LikhonSheikh. 2024. Banglanlp bengali-english parallel corpus.
- T. Magdeburg and K. Schultz. 2024. Investigating back-translation for bavarian-german nmt. *Machine Translation*, 38(1):45–67.
- M. Nguyen and D. Chiang. 2024. Comparative pipelines for low-resource transformer training. *Machine Translation*, 38(3):289–312.
- O. Obeid, M. Salameh, H. Bouamor, and N. Habash. 2024. Semi-supervised neural machine translation for egyptian arabic to modern standard arabic. *Natural Language Engineering*, 30(2):187–210.
- S. Ranathunga, E. Lee, M. Prifti Skenduli, R. Shekhar, M. Alam, and R. Kaur. 2023. Neural machine translation for low-resource languages: A survey. *ACM Computing Surveys*, 55(11):1–37.
- A. Smith, T. Brown, and M. Johnson. 2025. Transformer adaptations for low-resource scenarios. *Journal of AI Research*, 67(1):55–75.
- Y. Tomabechi, R. Kawakami, and M. Komachi. 2023. Machine translation for ainu language preservation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 445–457.
- H. Türkmen, A. Özgür, and T. Güngör. 2024. Efficient neural machine translation for turkish-english language pairs. *Applied Sciences*, 14(8):3215.
- J. Zhang, M. Utiyama, E. Sumita, G. Neubig, and S. Nakamura. 2018. Guiding neural machine translation with retrieved translation pieces. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 1325–1335.
- S. Zhang, G. Mahmut, D. Wang, and A. Hamdulla. 2017. Memory-augmented chinese-uyghur neural machine translation. In *2017 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference*, pages 1092–1096.